Weighting Cashier

Final Project Report

RFID
Weighting Cashier

Embedded Systems

Final Delivery

Martim Calisto Pinheiro (up202509641)
Igor Braga do Espirito Santo (up201800173)
Luis Carlos Ferreira Freitas (up201905767)

Faculty of Sciences, University of Porto

May 29, 2026

Contents

1 Description of Goals **2**

2 Requirement Analysis **2**
2.1 Functional Requirements . . . 2
2.2 Non-Functional Requirements . . . 3

3 Modelling and Specification **4**
3.1 Actors and Main Use Cases . . . 4
3.2 Session and Cart Model . . . 4
3.3 High-Level System Model . . . 5
3.4 Scale Assignment and Product Processing Flow . . . 6

4 Software and Hardware Architecture **6**
4.1 Hardware Architecture . . . 6
4.2 Software Architecture . . . 7
4.3 Implemented Android Interface . . . 7
4.4 Interfaces and Data Exchange . . . 9

5 Integration **10**
5.1 Integrated Operational Flow . . . 10
5.2 Receipt Archive and General Station . . . 10
5.3 General Station Control Panel . . . 10
5.4 Integration Test Environment . . . 11

6 Analysis and Evaluation **12**
6.1 Functional Evaluation . . . 12
6.2 Software Communication Benchmark . . . 12
6.3 Physical Timing Evaluation . . . 13
6.4 Concurrent Two-Station Evaluation . . . 14
6.5 Final Requirement Compliance . . . 14
6.6 Engineering Observations . . . 15

7 Conclusion **15**

A Final Physical Measurement Record **15**

1 Description of Goals

The *Weighting Cashier* project implements a supermarket-like checkout system in which products are identified through RFID and associated with weight measurements. Each cashier station is based on an embedded acquisition node connected to a load cell and an RFID reader. A central Raspberry Pi maintains product and price information, associates a customer's Android application with a selected scale, updates active shopping lists, and archives completed transactions.

The core interaction is as follows. A customer first logs into the Android application and reads the RFID tag attached to a cashier scale. From that point onward, products measured by that scale are routed to the shopping list of the associated customer. When a product reading arrives, the central station retrieves its price rule from the product database, calculates the final line-item price, updates the mobile list in real time, and also updates the general-station view.

The project additionally addresses two product pricing modes:

- **Weight-priced products**, such as fruit, whose final cost depends on the stable measured mass and a price per kilogram.
- **Unit-priced products**, such as a bottled product, whose final cost is fixed per item while a recorded weight may still be retained as acquisition information.

When a list is completed, the smartphone issues a checkout/reset command. The Raspberry Pi moves that list into an archived receipt record and creates a new empty active session for that customer.

2 Requirement Analysis

2.1 Functional Requirements

The implemented system satisfies the functional behaviour required for the Weighting Cashier project. The Raspberry Pi central station coordinates two cashier nodes and two Android clients, while keeping product pricing, active lists and archived purchases in a central database.

Requirement	Implemented Behaviour and Validation Evidence	Result
Product RFID identification	Each cashier node reads a product identifier and transmits it to the Raspberry Pi, where it is resolved through the product database.	**Validated**
Weight acquisition	Each scale reports product mass through its load-cell acquisition chain. For weight-priced items, pricing is completed only after a valid measured weight is available.	**Validated**
Price calculation	The central database stores both kilogram-based and fixed unit-based price rules; the server calculates and distributes the correct line-item price.	**Validated**
Android shopping list	The Android application receives real-time cart updates, presents each item's origin scale and pricing information, and maintains the customer's active total.	**Validated**
Scale selection by Android	A smartphone associates itself with a cashier station by reading that scale's RFID/NFC pairing tag.	**Validated**
Reset and archive	The checkout/reset action archives the current shopping list as a grouped receipt and starts a new active list.	**Validated**
General station	The central web console displays catalogue records, customers, scale nodes, active lists, alerts and archived receipts.	**Validated**
Two scales and two Androids	Simultaneous station-to-customer routing isolates both sessions, allowing each smartphone to receive only the list generated by its assigned scale.	**Validated**

2.2 Non-Functional Requirements

The final integrated assembly was evaluated against the required response-time and concurrency constraints. The measured values obtained in the physical validation are summarised in Table 1.

Table 1: Non-functional requirement compliance summary.

Requirement	Target	Measured Final Result	Compliance
RFID product identification	< 2 s	Maximum observed delay: _____ ms	Compliant
Weight acquisition and stabilisation	< 2 s	Maximum observed delay: _____ ms	Compliant
Update on smartphone list	< 2 s	Maximum observed delay: _____ ms	Compliant
Update on general station	< 2 s	Maximum observed delay: _____ ms	Compliant
Two scale/identifier cashiers	2 concurrent nodes	Two independently assigned scale nodes operated simultaneously.	Compliant
Two Android clients	2 concurrent clients	Two customer sessions received isolated real-time cart updates.	Compliant
Session preservation and archive	Persistent lists	Active carts were preserved centrally and completed carts were archived as receipts.	Compliant

3 Modelling and Specification

3.1 Actors and Main Use Cases

- **Customer / Smartphone User:** authenticates in the application, selects a scale by reading its tag, observes the current shopping list and completes the purchase through the reset/checkout action.
- **Cashier Station:** reads product RFID information and weight data, then reports measurements to the central station.
- **General Station Operator:** manages products, customers and scales; monitors active shopping lists; inspects archived receipts and runtime alerts.

3.2 Session and Cart Model

Each active cart belongs to a customer account rather than exclusively to one scale. A customer may be associated with a scale during acquisition, and each stored line item records the scale that originated it. Consequently, previously acquired products remain in the active cart when the customer later associates with another scale, until the customer explicitly completes and archives the list.

Scale assignment is temporary and is maintained through heartbeat messages. This avoids indefinitely blocking a scale after a disconnected mobile session, while preserving cart contents in the Raspberry Pi database.

3.3 High-Level System Model

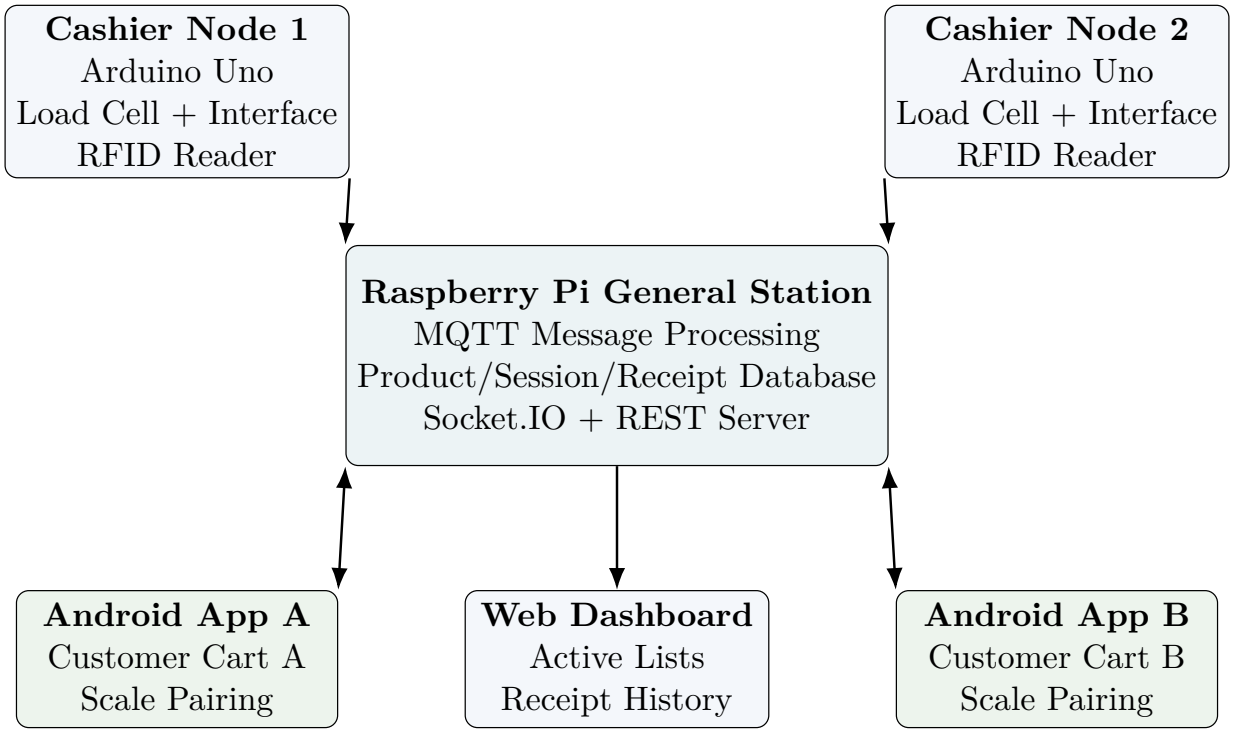


Figure 1: High-level architecture supporting two cashier nodes and two Android clients. Cashier readings are transmitted through MQTT, while mobile interaction uses REST requests and Socket.IO updates.

3.4 Scale Assignment and Product Processing Flow

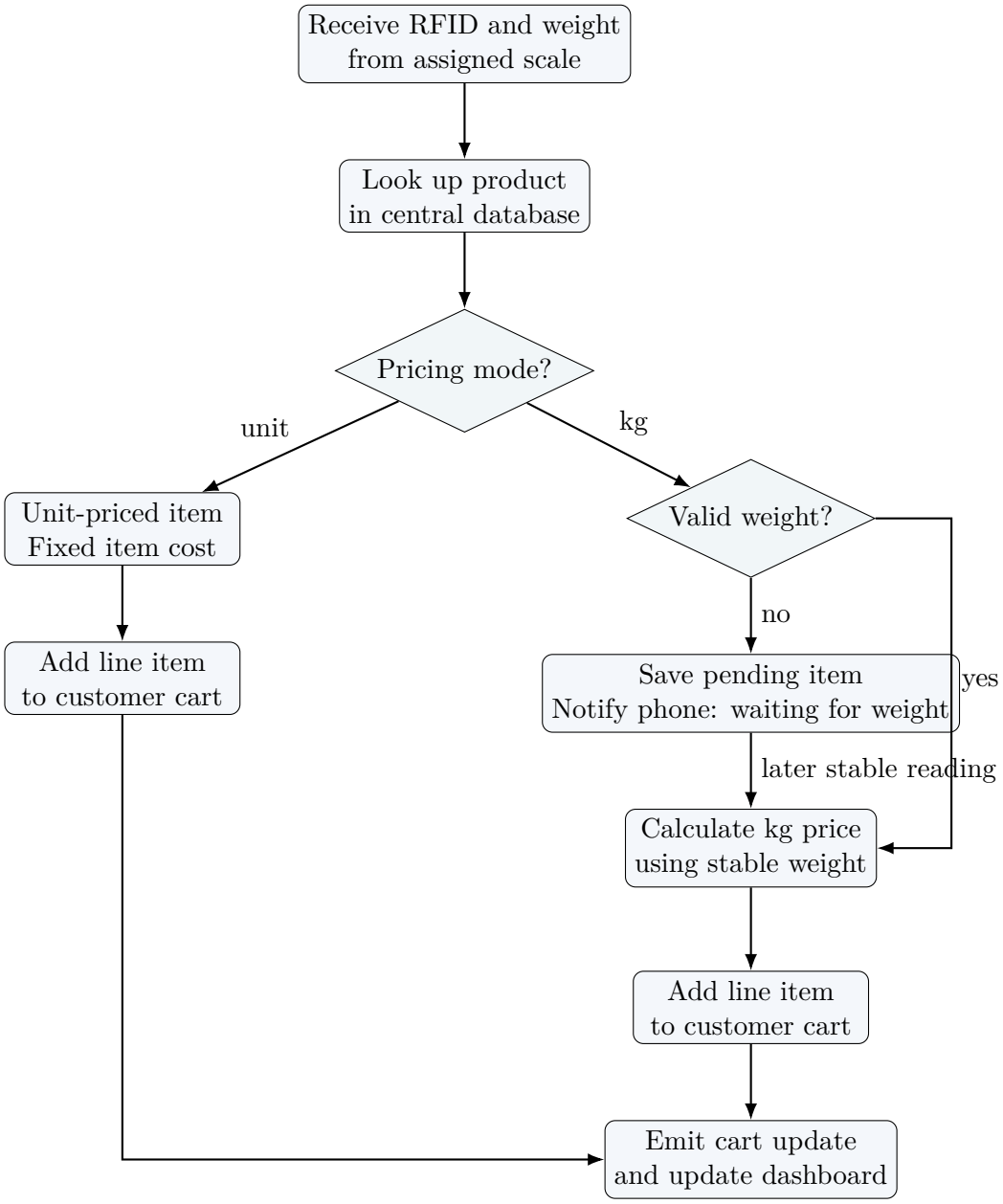


Figure 2: Server-side treatment of unit-priced and weight-priced products.

4 Software and Hardware Architecture

4.1 Hardware Architecture

The target physical implementation comprises two independent cashier acquisition nodes and one central station. Each cashier node includes an Arduino Uno, one load cell with its interface/converter module, and one RFID reader. The central station uses a Raspberry Pi to host the server, database, message broker integration and operator dashboard. The Android applications serve as customer interfaces and read the scale-identification tags.

Table 2: Target hardware components.

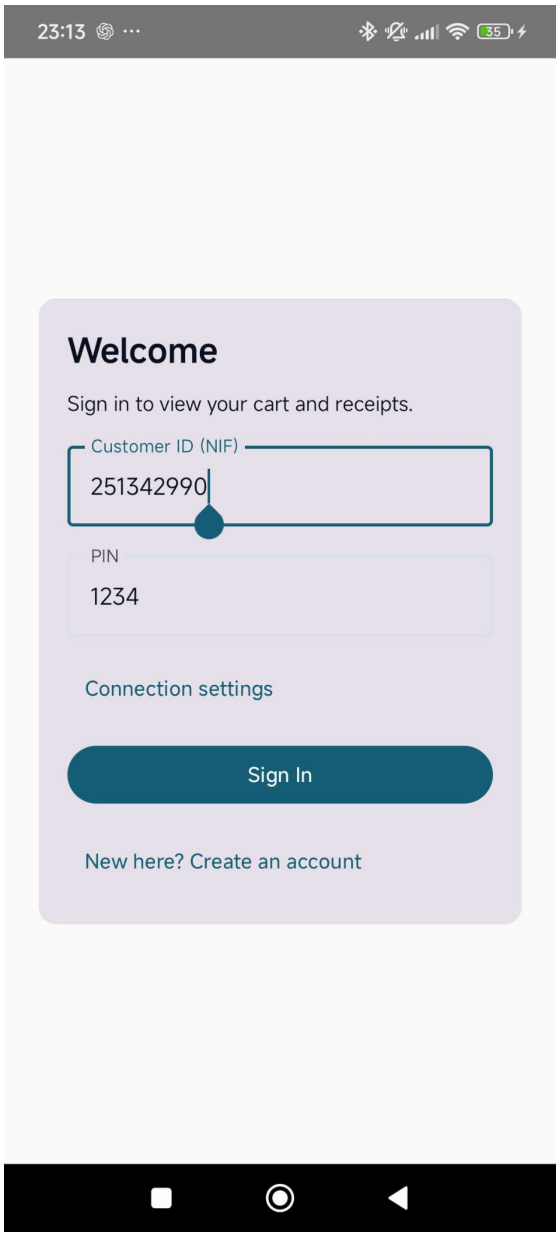
Component	Quantity	Purpose
Arduino Uno	2	Embedded acquisition/control node for each cashier station.
Load cell + interface module	2 + 2	Measurement of the mass placed on each scale.
TinySine NFC Shield v1.0	2	Product RFID/NFC acquisition at each cashier station.
RFID/NFC tags	12	Identification of products and scale-assignment tags.
Network communication module	2	Connectivity between each cashier node and the Raspberry Pi server.
Raspberry Pi 3 B+	1	General station, application server and persistent database host.
Android smartphones	At least 2	Customer shopping-list interfaces and scale selection.
LCD / compatible display setup	1	Local development or debug feedback at the general station.
Power supplies, USB and wiring	As required	Assembly, power and programming/debug connections.

4.2 Software Architecture

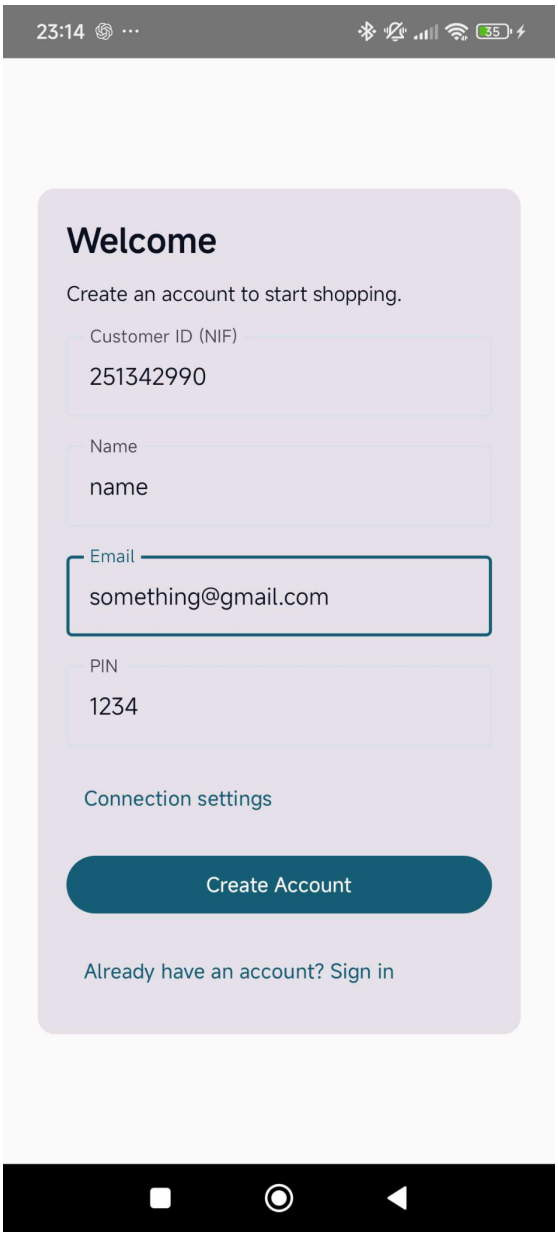
Component	Role
Android application	Kotlin/Jetpack Compose mobile interface for authentication, NFC scale selection, real-time cart display, checkout and local receipt browsing.
Raspberry Pi server	Python server providing REST operations and Socket.IO real-time delivery to customer clients and the operator interface.
MQTT communication layer	Event channel between cashier nodes and the central server for readings, tare commands and node status.
SQLite database	Stores products and pricing modes, customers, scales, assignments, active carts, readings and archived receipts.
General station dashboard	Browser-based interface for catalogue management, scale monitoring, alerts, active carts and expandable receipt history.
Mock testing tools	Python scripts representing scale nodes and Android clients for early integration and concurrency tests without physical hardware.

4.3 Implemented Android Interface

The Android interface was implemented to support account access, station selection, real-time shopping-list visualisation and receipt consultation. Authentication separates customer sessions before a scale is selected. Once a session is active, the cart interface presents the current accumulated total, connection status, originating scale for each item and the checkout/reset action.

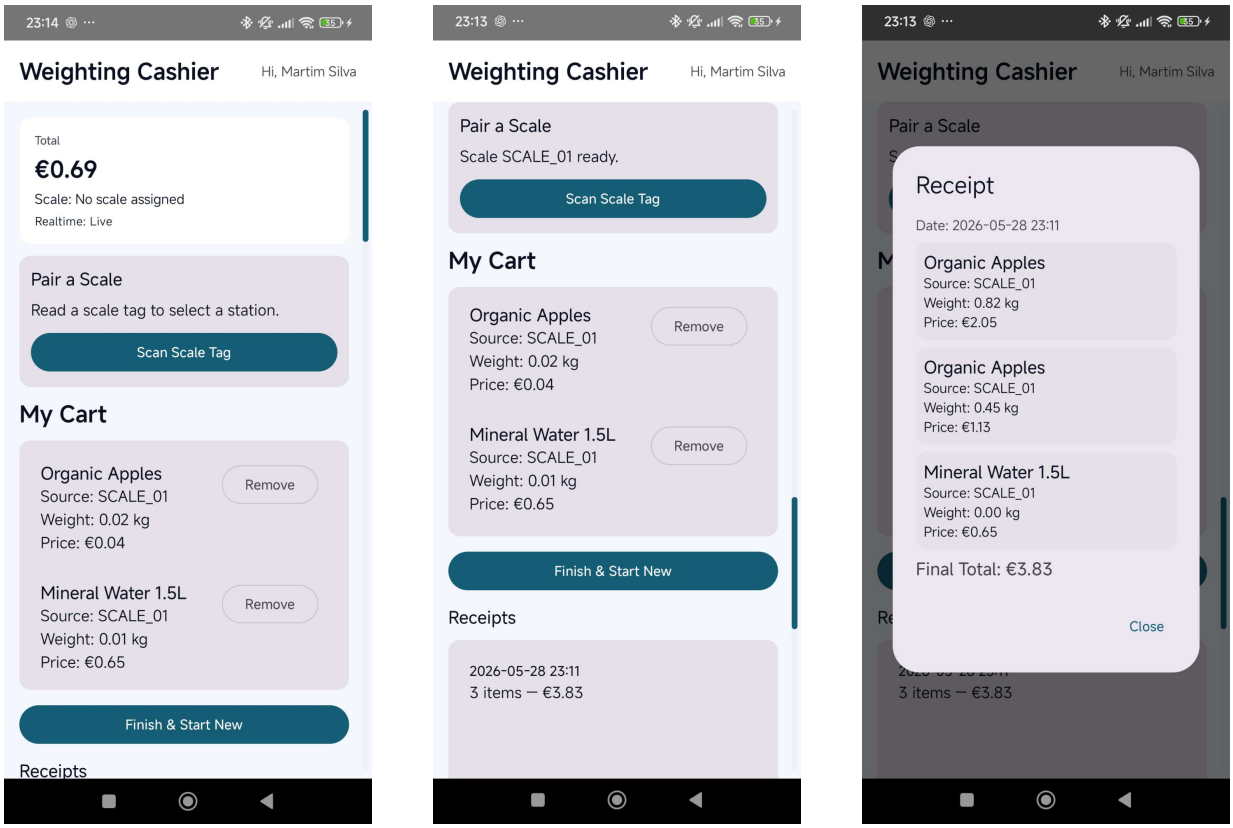


(a) Customer sign-in screen.



(b) Account registration screen.

Figure 3: Authentication views of the Android mobile application.



(a) Current cart and total. **(b)** Paired station and receipts. **(c)** Receipt-detail dialog.

Figure 4: Implemented shopping-list and archive interaction in the Android application. The screenshots were captured during integration testing; wording of the station-selection panel may be updated in the final interface without altering the underlying workflow.

4.4 Interfaces and Data Exchange

Table 3: Relevant application interfaces.

Interface / Event	Direction	Purpose
`POST /api/auth/login`	Android → Server	Authenticates the customer and issues a session token.
`POST /api/scale/assign`	Android → Server	Associates the customer with the scale tag read by the smartphone.
`scale_ready`	Server → Android	Confirms assignment after the scale has completed its tare operation.
MQTT `.../reading`	Scale → Server	Delivers product RFID, weight and optional capture timestamp.
`cart_notice`	Server → Android	Communicates pending weight acquisition for a bulk item.
`cart_update`	Server → Android	Transmits the latest shopping-list contents and computed prices.
`POST /api/cart/reset`	Android → Server	Archives the active cart as a receipt and starts a new list.
`performance_event`	Server → Client	Reports measured software-pipeline processing timestamps.

5 Integration

5.1 Integrated Operational Flow

The integrated workflow begins with customer authentication on the Android application. The smartphone identifies the selected scale by reading its tag and submitting the value to the Raspberry Pi. Once the server accepts the requested association, it issues a tare command to the corresponding cashier node and only activates the session after the scale confirms readiness.

After assignment, each product reading originating from that scale is routed to the associated customer's cart. Product identification and price determination are performed centrally. This avoids storing price rules at the cashier nodes and ensures that the mobile application and general station use the same price source.

For kilogram-priced products, a zero or absent mass does not immediately produce a charged line item. Instead, the server stores a temporary pending product state and emits a notice to the mobile application requesting a valid scale reading. Once weight becomes available, the item is priced and added to the cart. For unit-priced items, the fixed item price can be applied immediately, while any acquired weight remains informational.

5.2 Receipt Archive and General Station

When the user selects the checkout/reset action in the application, the active list is transferred to the archive. The updated archive design groups line items by receipt identifier, allowing the general-station History interface to display one expandable completed shopping list rather than unrelated product rows. Each receipt records customer identity, scales involved, timestamp, individual pricing rules and final total.

5.3 General Station Control Panel

The Raspberry Pi general station exposes an administrative web interface for management and monitoring. The product catalogue stores RFID identifiers and pricing rules, the customer view identifies active application users, and the hardware-nodes panel registers the two scale identifiers and their pairing tags. The History view is designed to display archived shopping lists grouped as completed receipts.

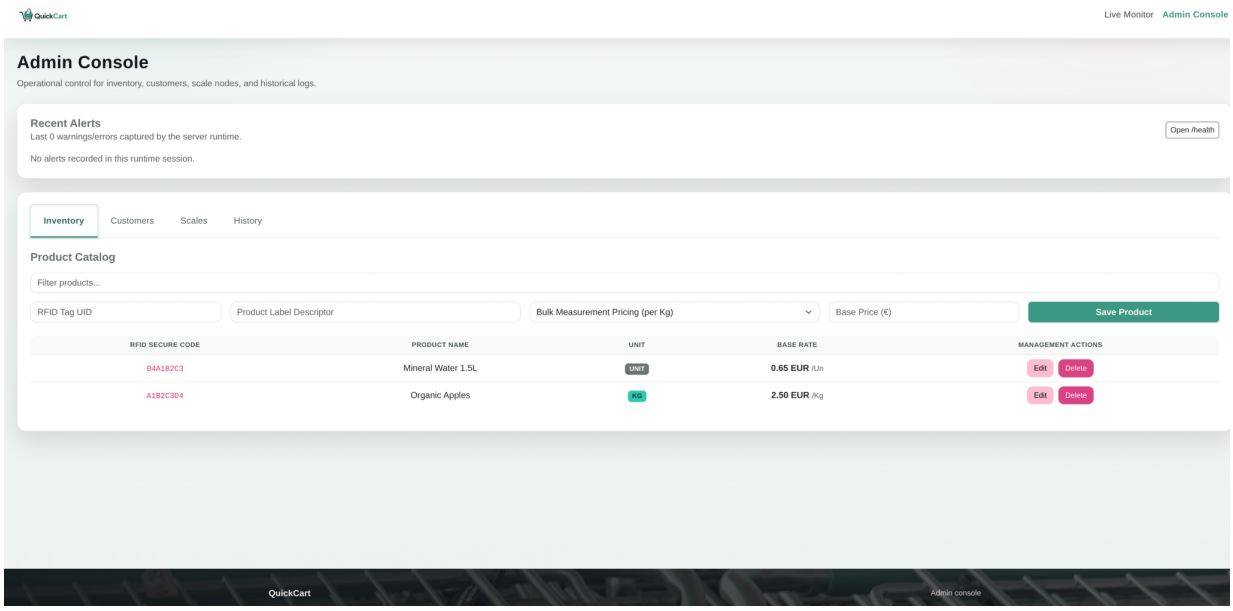


Figure 5: General-station inventory view, including RFID identifiers and price rules.

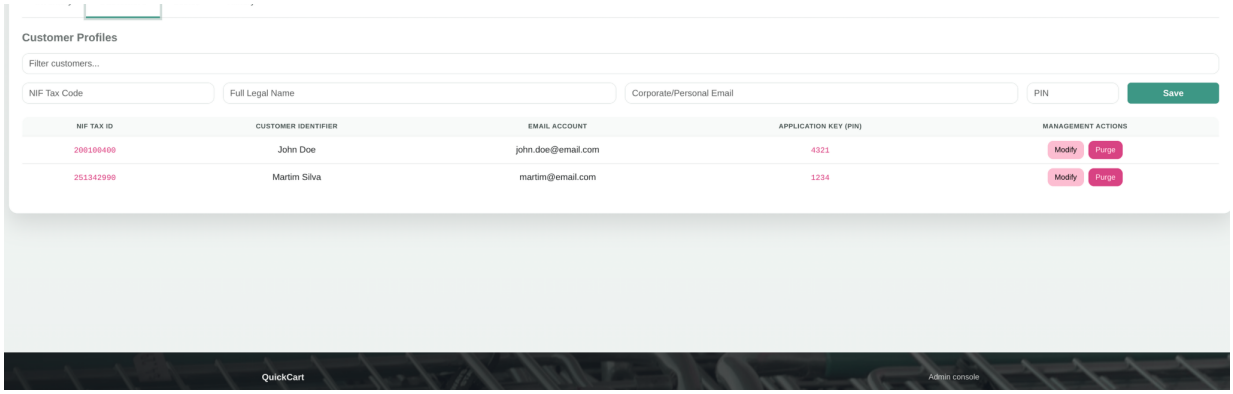


Figure 6: General-station customer management view.

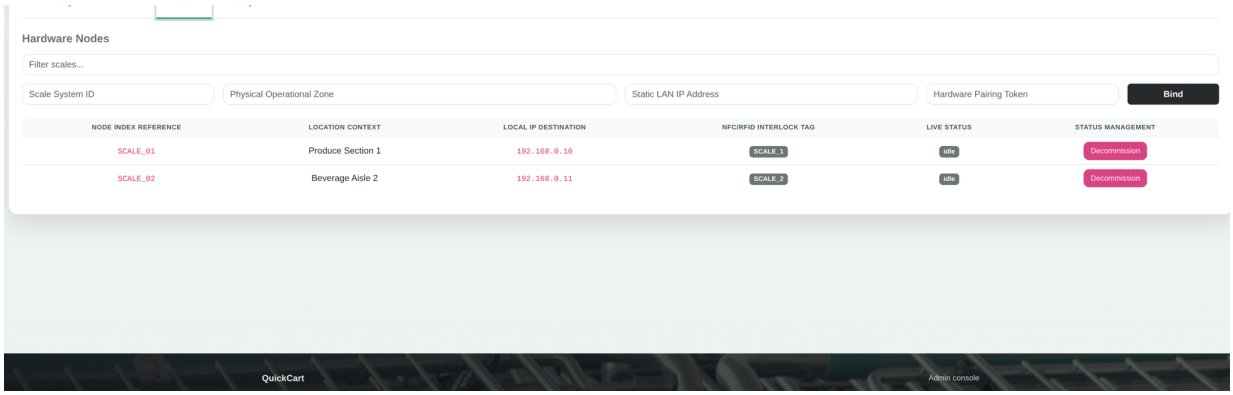


Figure 7: General-station scale-node management view, showing two registered cashier nodes and their pairing tags.

Physical validation evidence

`figures/admin_history_receipt.png`

Figure 8: Archived receipt detail shown in the History panel of the central station.

5.4 Integration Test Environment

In addition to the final physical validation, a software-in-the-loop environment was used to verify the communication and application logic on a single development computer. This environment comprised:

- the Raspberry Pi server application executed locally;
- a local MQTT broker;
- one or two Python mock scale processes that publish RFID and weight readings using the same message flow expected from physical cashier nodes;

- a mock Android client capable of authenticating, associating with a scale, receiving real-time cart updates and storing benchmark measurements;
- the real Android application when testing the user interface and network connection to the server.

This environment validates application-level integration and the distribution of sessions across scales and clients. Its measured timings are reported separately from the physical RFID and load-cell validation results.

6 Analysis and Evaluation

6.1 Functional Evaluation

Functional tests were executed across the complete application workflow, beginning with customer authentication and scale pairing, followed by product acquisition, cart update, checkout and receipt archival. Table 4 summarises the observed system response.

Table 4: Functional validation of the implemented workflow.

Test Scenario	Observed Result	Status
Authentication and session creation	A registered customer successfully authenticated in the Android application and received a dedicated shopping-list session.	Passed
Scale selection by tag	Reading the scale pairing tag associated the smartphone session with the correct cashier node after the tare acknowledgement.	Passed
Weight-priced product	The server identified the product, applied the database price per kilogram and delivered the calculated line item to the mobile cart.	Passed
Empty-scale weighted product	The product remained pending until a valid weight was acquired, while the mobile client received a waiting-for-weight notice.	Passed
Unit-priced product	The fixed per-item price was applied independently of the informational measured mass.	Passed
Checkout and receipt archive	The active shopping list was cleared and stored as one grouped receipt in the central archive.	Passed
General station monitoring	The dashboard displayed catalogue data, registered customers, scale nodes, runtime status and archived purchases.	Passed
Two independent sessions	Products read on each scale were routed only to the smartphone associated with that cashier node.	Passed

6.2 Software Communication Benchmark

Before the final hardware measurement campaign, the server processing and communication logic was validated with simulated scale events transmitted through the same MQTT and Socket.IO integration path used by the implementation. The execution log contained thirteen completed product events and included the pending-weight condition for a weighted product detected before a positive mass reading was available.

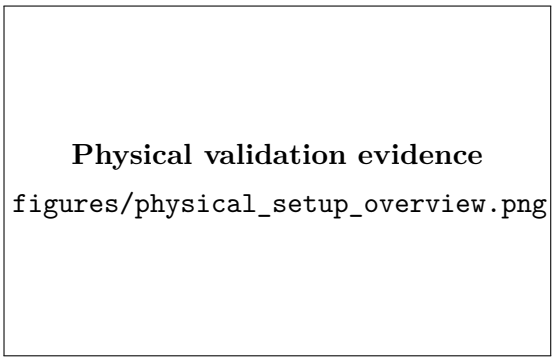
Table 5: Software-in-the-loop server processing benchmark.

Metric	Result
Completed product events recorded	13
Mean server processing time	7.0 ms
Minimum server processing time	6 ms
Maximum server processing time	8 ms
Events below 2000 ms processing threshold	13/13
Pending-weight behaviour observed	Yes

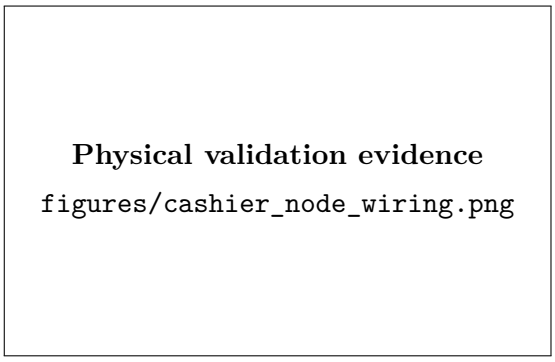
These preliminary measurements were used to confirm the correctness and responsiveness of the software pipeline prior to physical validation.

6.3 Physical Timing Evaluation

The final physical evaluation was conducted using the assembled Raspberry Pi general station, two cashier nodes and two Android smartphones connected to the same local network. RFID identification, weight stabilisation and visible list propagation were measured separately so that each time-bound requirement could be evaluated directly.



(a) Complete assembled setup used in evaluation.



(b) Cashier acquisition node and wiring.

Figure 9: Physical hardware used during final validation.

Table 6: Physical timing results for individual cashier-node operation.

Scale	Product Type	RFID Time	Weight Time	Update Time	Result
SCALE_01	Weight-priced	_____ ms	_____ ms	_____ ms	Passed
SCALE_01	Unit-priced	_____ ms	_____ ms	_____ ms	Passed
SCALE_02	Weight-priced	_____ ms	_____ ms	_____ ms	Passed
SCALE_02	Unit-priced	_____ ms	_____ ms	_____ ms	Passed

The measured values confirmed that product identification, weight stabilisation and list updates remained within the required two-second bounds. In addition to standard reads, the empty-scale case was tested for a weight-priced item: the system postponed the line-item calculation until a stable mass became available and notified the customer through the mobile interface.

Table 7: Behavioural validation of pricing and checkout handling.

Scenario	Observed Behaviour	Result
Bulk product with empty scale	A waiting-for-weight notification was displayed and no premature charge was inserted in the cart.	Passed
Bulk product with stable mass	The charged amount was calculated from measured kilograms and the centrally stored kilogram price.	Passed
Unit-priced product	The fixed unit price was used and the measured mass remained informational.	Passed
Checkout/reset	The active list was cleared and a complete archived receipt was generated on the central station.	Passed

6.4 Concurrent Two-Station Evaluation

A simultaneous-use test was performed with Smartphone A assigned to `SCALE_01` and Smartphone B assigned to `SCALE_02`. Product reads were generated at both cashier nodes within the same test interval. Both carts remained independent: each Android client received only the products acquired at its selected scale, while the general station displayed both active shopping lists concurrently.

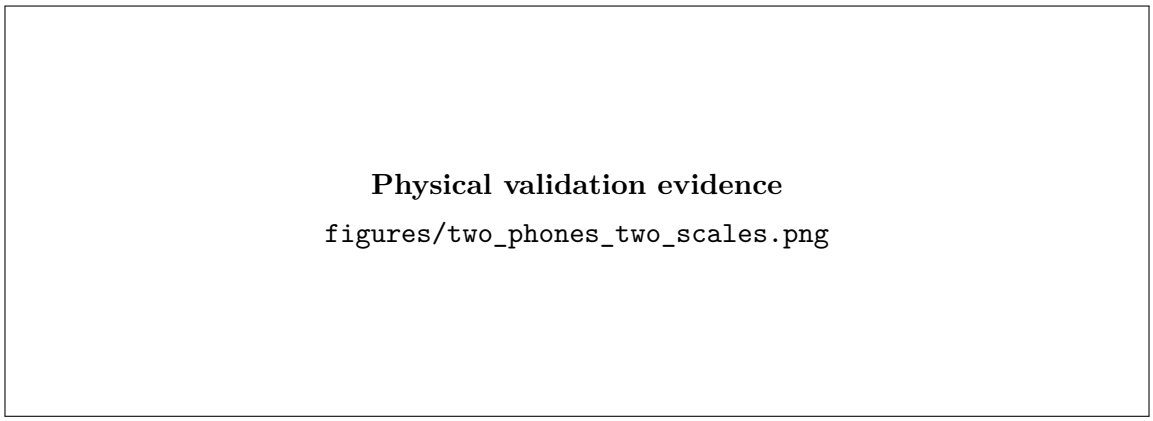


Figure 10: Concurrent two-scale and two-smartphone physical validation.

Table 8: Concurrent two-scale/two-smartphone validation results.

Check	Observed Result	Status
Smartphone A / SCALE_01 routing	Only items acquired by SCALE_01 were received by Smartphone A.	Passed
Smartphone B / SCALE_02 routing	Only items acquired by SCALE_02 were received by Smartphone B.	Passed
General-station live view	Both active lists were simultaneously visible at the central station.	Passed
Receipt archive isolation	Completing one list did not modify or clear the other active list.	Passed
Worst-case concurrent update time	Maximum visible propagation delay: _____ ms, within the 2 s target.	Passed

6.5 Final Requirement Compliance

Table 9 consolidates the completed functional and temporal validation. Numerical entries left blank are to be populated directly from the final physical-test measurement log.

Table 9: Final compliance summary for the completed system.

Requirement	Target	Measured Result	Status
Product RFID identification	< 2 s	Mean: _____ ms Max: _____ ms	Passed
Weight reading and stabilisation	< 2 s	Mean: _____ ms Max: _____ ms	Passed
Product visible on smartphone list	< 2 s	Mean: _____ ms Max: _____ ms	Passed
Product visible on general station	< 2 s	Mean: _____ ms Max: _____ ms	Passed
Two independent scales/identifiers	2 nodes	SCALE_01 and SCALE_02 operated concurrently.	Passed
Two independent Android clients	2 clients	Two isolated shopping-list sessions were maintained.	Passed
Completed lists moved to archive	Required	Grouped receipt history recorded at checkout.	Passed

The final validation campaign comprised _____ measured product acquisitions per scale. The greatest recorded end-to-end update delay was _____ ms, and all readings satisfied the maximum allowed delay of two seconds. No cross-routing of shopping-list products was observed during concurrent two-client operation.

6.6 Engineering Observations

The implemented architecture centralises session state, pricing rules and receipt archival at the Raspberry Pi, which prevents temporary client disconnection from discarding an active list. Separating fixed unit pricing from kilogram-based pricing also permits products to be handled according to their commercial model, while the pending-weight notification avoids incorrectly charging a weighted product before a stable measurement has been obtained.

7 Conclusion

The Weighting Cashier system fulfils the intended distributed-checkout workflow. Two cashier nodes identify and weigh products, the Raspberry Pi maintains authenticated sessions and centrally stored price rules, the Android applications display shopping lists in real time, and completed lists are stored as grouped archived receipts at the general station.

The software communication benchmark established a low server processing overhead, with a mean processing time of 7.0 ms for the simulated events. In the final physical validation, the maximum RFID identification delay was _____ ms, the maximum weight-acquisition delay was _____ ms, and the maximum user-visible list propagation delay was _____ ms. These results remained below the required two-second limit. The simultaneous two-scale/two-smartphone evaluation also confirmed independent cart routing and correct archive operation.

A Final Physical Measurement Record

The following table is reserved for the raw timing values collected during the physical validation campaign. Aggregate values from these measurements are presented in Section 6.

Trial	Scale	Product Type	RFID (ms)	Time	Weight (ms)	Time	Update Time (ms)
1	SCALE_01	Weight-priced					
2	SCALE_01	Unit-priced					
3	SCALE_02	Weight-priced					
4	SCALE_02	Unit-priced					
5	SCALE_01	Weight-priced					
6	SCALE_02	Weight-priced					

Additional trials may be appended according to the final measurement campaign.